

Universitatea „Sapientia” din Cluj-Napoca  
Facultatea de Științe Tehnice și Umaniste din Târgu-Mureș  
Specializarea Calculatoare

# **Bus4U - UI**

## **PROIECT DE DIPLOMĂ**

**Coordonator științific:**

Ș.l.dr.ing. Szántó Zoltán

**Absolvent:**

Portik Szabolcs

2024

Sapientia Erdélyi Magyar Tudományegyetem  
Marosvásárhelyi Kar  
Számítástechnika szak

# **Bus4U - UI**

## **DIPLOMADOLGOZAT**

**Témavezető:**

Dr. Szántó Zoltán

Egyetemi adjunktus

**Hallgató:**

Portik Szabolcs

2024

## Kivonat

Napjaink felgyorsult világában kevesen használják a tömegközlekedést, mert az nem elég kényelmes, nem elég gyors vagy éppen nem úgy működik, ahogy azt elvárnánk tőle. A gyakori problémák közé tartozik, hogy a buszok nem tartják a menetrendet vagy a jegyek megvásárlása nehézkes. Sok helyen a menetrend nincs kifüggesztve a megállóknál és az interneten sem elérhető. Az is gyakran megesik hogy a buszok nagyon zsúfoltak és erről az utazni vágyók csak akkor vesznek tudomást, amikor már a megállóban vannak és nem áll rendelkezésre más utazási lehetőség. A Bus4U online platform ezen problémákra kínál megoldást.

Az Bus4U egy mobilalkalmazásból, egy felhasználói weboldalból és egy adminisztrátori weboldalból tevődik össze. Főbb funkciói közé tartozik a buszjáratok keresése a kiindulópont, a célállomás és az időpont megadásával, a járatokra szóló előzetes online jegyvásárlás, illetve a járatok indulási időpontjainak megtekintése városokra és megállókra lebontva. Ezen kívül megtekinthetők a buszmegállók a térképen, városok és útvonalak szerint szűrve, azért, hogy a felhasználó ki tudja választani a legoptimálisabb helyet a felszálláshoz. A live map funkcióban valós időben követhetők az autóbuszok pillanatnyi helyzetei, hogy a felhasználó előre informálódni tudjon a menetrend változásáról és az esetleges pontatlanságokról. Ezen funkciók biztosításához, illetve a jegykezeléshez, szükség van egy alkalmazásra, ami a buszokon található POS terminálon fut. Ez biztosítja a jegyek érvényesítését, illetve az autóbuszok élő információinak megosztását is a központi szerverrel.

Ezen felül a webalkalmazás biztosít egy adminisztrációs felületet is a busztársaságok számára. Itt lehetőség nyílik elsősorban a menetrendek és a megállók kezelésére, valamint az útvonalak feltöltésére. A busztársaság adminisztrátora itt tudja kezelni az alkalmazottak információit, és az autóbuszok különböző adatait is, mint az időszakos műszaki vizsga és a biztosítás. Végül pedig itt található egy statisztikai felület is, ahol a busztársaságok megtekinthetik a forgalmukat, a regisztrált felhasználók számát, az eladott jegyek számát és az ezekből generált bevételt, mindezt havi, éves és mindenkori bontásban.

Továbbfejlesztési lehetőségek között szerepelnek a bérletek, illetve a különböző kedvezmények létrehozásának lehetősége, valamint a buszok telítettségének követése valós időben. A busztársaságok szempontjából fontos lehet még a sofőrök munkaidejének követése és az automatizált könyvelés bevezetése is.

Ez a dolgozat a szoftver felhasználói felületére fókuszál, a két weboldalra és a mobilalkalmazásra, míg a backend működése egy másik dolgozat keretein belül van ismertetve.

**Kulcsszavak:** tömegközlekedés, buszjárat, digitalizálás

# Tartalomjegyzék

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>1. Célkitűzések</b>                             | <b>4</b>  |
| <b>2. Szakirodalmi tanulmány</b>                   | <b>5</b>  |
| 2.1. Létező megoldások . . . . .                   | 5         |
| 2.1.1. Moovit . . . . .                            | 6         |
| 2.1.2. Budapest Go . . . . .                       | 6         |
| <b>3. A rendszer specifikációi</b>                 | <b>7</b>  |
| 3.1. Felhasználói követelmények . . . . .          | 7         |
| 3.1.1. Felhasználói felület . . . . .              | 8         |
| 3.1.2. Adminisztrátori felület . . . . .           | 8         |
| 3.2. Rendszerkövetelmények . . . . .               | 10        |
| 3.2.1. Funkcionális követelmények . . . . .        | 10        |
| 3.2.2. Nem funkcionális követelmények . . . . .    | 11        |
| 3.3. A rendszer architektúrája . . . . .           | 12        |
| 3.3.1. Frontend . . . . .                          | 13        |
| 3.3.2. Backend . . . . .                           | 13        |
| <b>4. A felhasználói felület tervezése</b>         | <b>14</b> |
| 4.1. Material Design . . . . .                     | 14        |
| 4.2. Figma terv . . . . .                          | 15        |
| <b>5. Gyakorlati megvalósítás</b>                  | <b>18</b> |
| 5.1. Felhasznált technológiák . . . . .            | 18        |
| 5.1.1. Vue.js . . . . .                            | 18        |
| 5.1.2. Vuetify . . . . .                           | 20        |
| 5.1.3. Flutter . . . . .                           | 20        |
| 5.2. Felhasználói weboldal és alkalmazás . . . . . | 21        |
| 5.2.1. Főoldal . . . . .                           | 21        |
| 5.2.2. Jegyek . . . . .                            | 24        |
| 5.2.3. Menetrend . . . . .                         | 25        |

|           |                                           |           |
|-----------|-------------------------------------------|-----------|
| 5.2.4.    | Megállók és élő térkép . . . . .          | 26        |
| 5.3.      | Adminisztrátori weboldal . . . . .        | 26        |
| 5.3.1.    | Kezdőoldal/Statisztikák . . . . .         | 26        |
| 5.3.2.    | Megállók . . . . .                        | 27        |
| 5.3.3.    | Útvonalak . . . . .                       | 28        |
| 5.3.4.    | Menetrend . . . . .                       | 29        |
| 5.3.5.    | Buszok és alkalmazottak . . . . .         | 29        |
| <b>6.</b> | <b>Összefoglalás</b>                      | <b>31</b> |
| 6.1.      | További fejlesztési lehetőségek . . . . . | 31        |

# 1. fejezet

## Célkitűzések

Fő célunk egy olyan rendszer kialakítása, amely egyaránt hasznos a tömegközlekedést választó embereknek, illetve a busztársaságoknak. A végfelhasználó szempontjából egy olyan weboldal, illetve mobilalkalmazás elkészítése a cél, amelyben meg lehet tekinteni a menetrendet, jegyet lehet vásárolni a járatokra, meg lehet tekinteni ezek útvonalát és megállóit, valamint élőben lehet követni a buszok jelenlegi helyzetét, hogy tudni lehessen mikor érnek a hozzánk legközelebb eső megállóba.

A busztársaságok szempontjából egy webes admin felület létrehozása a cél, ahol az adminisztrátorok teljeskörűen menedzselhetik cégük működését, egyéb szoftverek bevonása nélkül. Itt az adminisztrátor feltöltheti a cég járműveinek adatait és az útvonalaikat, megjelölheti a térképen a megállóikat, szerkesztheti a menetrendeket, és a jegyárakat is beállíthatja. Ugyanezen a platformon egy összegzést találhat a profitról, a felhasználók számáról illetve az eladott jegyekről, különböző időintervallumokra leosztva.

Nem utolsó sorban egy második, kisebb alkalmazás létrehozása is a célok közé tartozik, amely a buszon lévő androidos POS terminálon fog futni, és ez szolgáltatja a buszok valós idejű helyzetét, illetve ezen lehet beszkenyelni a jegyet a buszra való felszálláskor.

## 2. fejezet

# Szakirodalmi tanulmány

A folyamatos technológiai fejlődés hatására egyre nagyobb szükség van a tömegközlekedés digitalizálására is. Az idők során egyre kényelmesebb és elérhetőbb lett a személyautóval való közlekedés, de sajnos ez nem mondható el a tömegközlekedésről. Ennek az az eredménye, hogy egyre többen választják a saját autójukat a tömegközlekedés helyett, ami a városi közlekedés túlterheltségéhez és a levegő erőteljesebb szennyezéséhez vezet. A tömegközlekedés digitalizálása segíthet a közlekedési hálózatok hatékonyságának növelésében és a környezetszennyezés csökkentésében is.

A tömegközlekedés megítélésének javításához szükség lenne egy olyan rendszerre, amely valós idejű információkat szolgáltat a járatokról, lehetővé teszi az online jegyvásárlást és az útvonaltervezést. Ehhez a buszok, vonatok és metrók különböző eszközökkel való felszerelésére van szükség, mint a jegyszkenelő automata, a kártyás fizetést biztosító POS terminál, a GPS vagy a forgalmat felmérő kamerák. Egy ilyen rendszerrel való interakcióba lépéshez szükség van egy felhasználói felületre is, amely lehetővé teszi az utasok számára a különböző szolgáltatások elérését: az útvonaltervezést, a valós idejű forgalmi adatok megtekintését és a jegyvásárlást [4]. Erre a célra már léteznek megoldások, de ezek nem fedik le teljesen a busztársaságok igényeit és még messze állunk attól, hogy ezt minden város vagy busztársaság implementálni tudja.

### 2.1. Létező megoldások

Kutatásunk során többek között rátaláltunk a Moovit és a Budapest Go nevű alkalmazásokra, amelyek két különböző megközelítést alkalmaznak a városi közlekedés digitalizálására, de céljaink hasonlóak: hogy a rendszer egyetlen platformon biztosítsa a tömegközlekedéshez kapcsolódó különféle szolgáltatásokat, beleértve a jegyvásárlást, útvonaltervezést és a valós idejű járatinformációk megjelenítését.

### **2.1.1. Moovit**

A Moovit egy népszerű alkalmazás, amely a világ számos országában és városában működik, így egy szinte mindenhol elérhető szolgáltatást nyújt, viszonylag sok funkcióval. Az alkalmazás lehetővé teszi az utasok számára, hogy megtervezzék az útvonalukat, figyelemmel kísérik a járatokat és megvásárolják a jegyeket. A közlekedési információkat az utasok okostelefonjaikon futó alkalmazások biztosítják, amelyek indóknént beküldik a GPS pozícióikat a Moovit szervereire. Ezenkívül a felhasználók manuálisan is megadhatnak információkat, például a járatok késéséről vagy pillanatnyi telítettségéről. A rendszer ezeket az adatokat dolgozza fel és jeleníti meg a többi felhasználó számára, amennyiben ezek elegendő mennyiségben rendelkezésre állnak. Ezzel csak az a probléma hogy kevés országban éri el a felhasználók száma az elvárt szintet, így ezek a funkciók nem mindenhol érhetőek el [4].

Sajnos a kevés felhasználót számláló országok közé tartozik a mienk is, így csak a hivatalosan megadott adatokat jeleníti meg a Moovit, a felhasználóktól érkezőket nem, emiatt nálunk eléggé rövid az alkalmazás által nyújtott szolgáltatások listája. Marosvásárhelyen például csak a városi buszok menetrendje és útvonala érhető el, de ehhez sem társul élő forgalominformáció és jegyvásárlási lehetőség sem.

### **2.1.2. Budapest Go**

A másik alkalmazás a Budapest Go, amely egy általános közlekedési rendszer és a városi közlekedés minden elemét lefedi, így nem csak a buszokkal való közlekedésre használható, hanem majdnem az összes tömegközlekedési eszközzel a metrótól, a villamoson át, a hajóig, sőt az egyéni közlekedés megkönnyítése végett a gyalogos- és bicikliútvonalak is megtalálhatóak benne. Ez a megoldás talán közelebb áll a mi elképzeléseinkhez a funkcionalitás szempontjából, mert ugyanúgy biztosítja a jegyvásárlás lehetőségét, az útvonaltervezést és a valós idejű járatinformációkat, mint a Moovit, de a Budapest Go esetében ezek a funkciók sokkal részletesebbek és szélesebb körűek, emellett pedig rengeteg egyéb funkciót is biztosít.

Egy lényeges probléma a Budapest Go-val az, hogy nem használható fel univerzálisan, mert ez a Budapesti Közlekedési Központ (BKK) saját rendszere, ezáltal pedig Budapestre van korlátozva és csak az ottani utasok tudják kihasználni az általa biztosított szolgáltatásokat [5]. Ezzel szemben mi a nyitottságra szeretnénk fektetni a hangsúlyt és egy olyan rendszert hoznánk létre, amelyet bármely busztársaság tudna használni, helytől függetlenül.



## 3. fejezet

# A rendszer specifikációi

A rendszer két fő részre bontható fel: backend és frontend. Mivel én a frontend résszel foglalkoztam, amely magában foglalja a felhasználói weboldalt, az adminisztrátori weboldalt és a mobil applikációt, ezért a következőkben csak ezekre a részekre vonatkozó követelmények specifikációját fogom részletezni.

### 3.1. Felhasználói követelmények

A felhasználói követelmények azokat a funkciókat írják le, amelyre a felhasználó szemszögéből szükség van. A szoftver általános felhasználói követelményei a következők:

- A felhasználó gyorsan és egyszerűen kell tudjon navigálni az oldalak között.
- Minden felhasználói bevitel esetén a rendszernek biztosítani kell a kényelmes használatot, valamint a megadott adatok helyességéről visszajelzést kell biztosítani beszédes hibaüzenek megjelenítése által.
- A felhasználói felületnek reszponzívnak kell lennie, azaz minden eszközön jól használhatónak kell lennie, legyen az mobiltelefon, tablet vagy számítógép.
- A felhasználói felületnek egységesnek kell lennie, azaz minden oldalon ugyanazok a színek és felületi elemek jelennek meg, hogy a felhasználók ne érezzék zavarónak az oldalak közötti váltást.

A frontend három különböző felületet tartalmaz: mobilalkalmazás, felhasználói- és adminisztrátori weboldal, ezek pedig különböző felhasználói követelményekkel rendelkezhetnek. A mobilapplikáció ugyanúgy működik, mint a felhasználóknak szánt weboldal, így ezeknek a követelményeit együtt fogom tárgyalni felhasználói felület néven. Ezzel szemben az adminisztrátori weboldal különálló egység, amelynek saját követelményei vannak.

### 3.1.1. Felhasználói felület

A felhasználói felület funkcióinak nagy része bejelentkezés nélkül használható. A fiókot igénylő funkciók a jegyvásárlás és a jegyek listázása. Az ezek eléréséhez szükséges regisztráció és bejelentkezés bármely oldalról elérhető. A felhasználói felület funkciói oldalanként csoportosítva a következők:

- Főoldal: járatokra való rákeresés dátum, idő, kezdő- és célállomás alapján, találatok listázása indulási idő szerinti növekvő sorrendben, jegyvásárlás (bejelentkezés szükséges)
- Menetrendek: buszok és megállók órarendjeinek megtekintése, járatok útvonalának megjelenítése térképen
- Megállók: megállók listázása térképen, szűrés település vagy buszjárat alapján
- Élő térkép: buszok valós idejű pozícióinak megtekintése térképen
- Jegyek (bejelentkezés szükséges): megvásárolt jegyek listázása QR-kódokkal

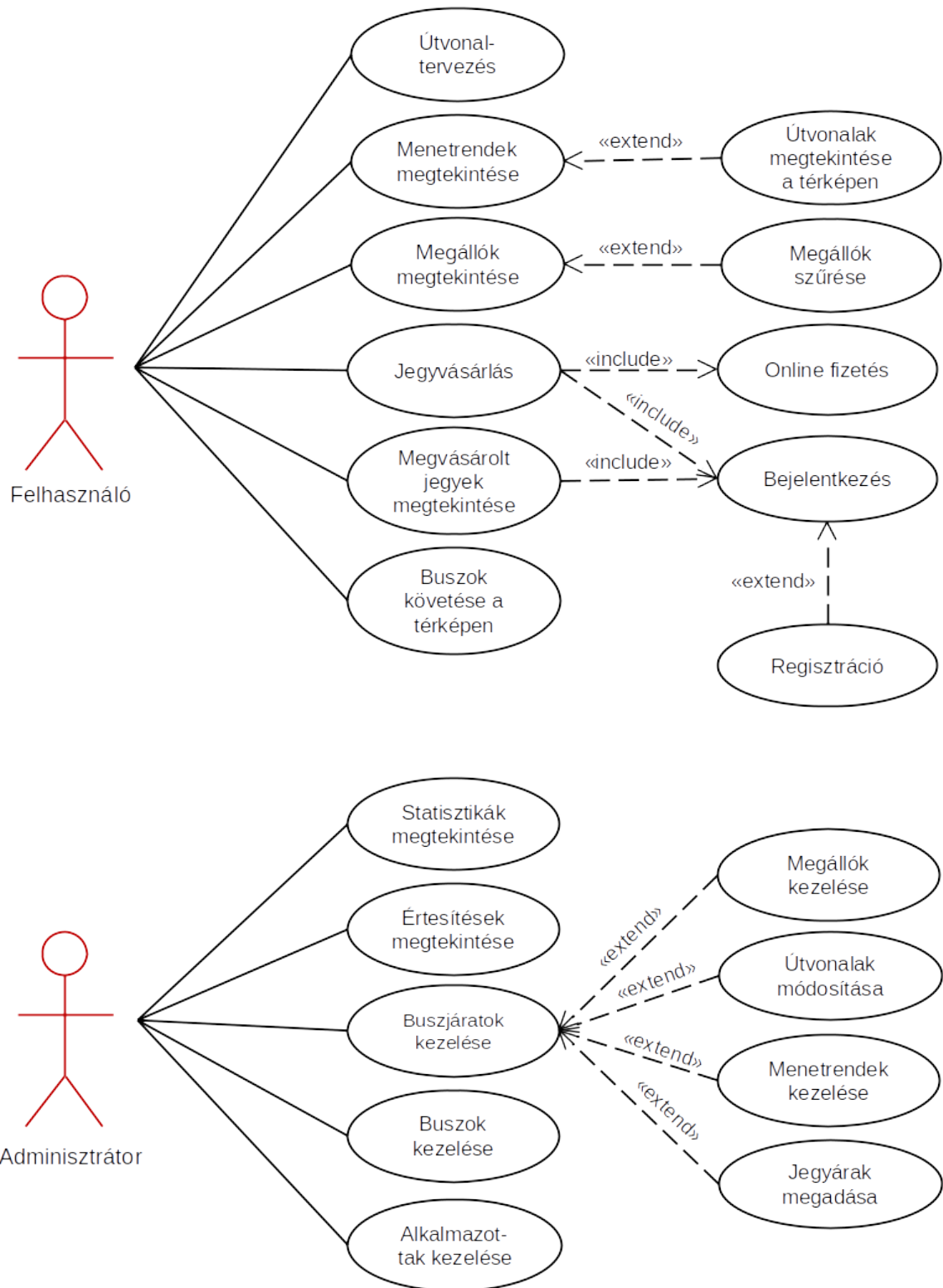
### 3.1.2. Adminisztrátori felület

Az adminisztrátori felület kizárólag bejelentkezéssel elérhető, új fiókokat pedig csak a cégek főnökei tudnak létrehozni, mindenik a saját busztársaságához. A társult vállalatok főnökei a partnerség kezdetekor megkapják az admin jogú fiókjaikat. Az adminfelület funkciói oldalanként csoportosítva:

- Főoldal: statisztikák az eladott jegyek számáról, a megtett kilométerről és a bevételről, időintervallumok szerint csoportosítva, értesítések megjelenítése
- Megállók oldal: megállók listázása, új megálló hozzáadása térkép segítségével, meglévő megálló szerkesztése, megállók törlése
- Járatok oldal: buszjáratok listázása, új járat hozzáadása, meglévő járat szerkesztése: megállók hozzáadása és eltávolítása, járatok törlése
- Menetrendek oldal: járatok indulási időpontjainak és jegyárainak megtekintése, szerkesztése és törlése
- Buszok oldal: buszok listázása, új busz hozzáadása, meglévő busz szerkesztése: útdó, biztosítás és műszaki vizsga időpontok megadása, buszok törlése
- Alkalmazottak oldal: alkalmazotti fiókok listázása, új alkalmazott hozzáadása, meglévő alkalmazott szerkesztése, alkalmazottak törlése

Belépés előtt bármely oldalra rákeresve, a rendszer átirányítja a felhasználót a bejelentkező felületre. Regisztrációs lehetőség nincs, így meg lehet akadályozni az adminfelülethez való illetéktelen hozzáférést.

A teljes rendszer funkcionalitását leíró használati eset diagram a 3.1-es ábrán látható.



3.1. ábra. Use-case diagram a felhasználók és az adminisztrátorok lehetőségeiről

## 3.2. Rendszerkövetelmények

### 3.2.1. Funkcionális követelmények

A funkcionális követelmények a felületen megjelenő funkciók működését írják le. Ezek foglalkoznak össze, hogy adott bemenetre milyen kimenetet kell szolgáltatnia a rendszernek. A Bus4U esetében ezeket is két csoportba sorolhatjuk: felhasználói és adminisztrátori követelmények.

A felhasználói weboldalon és a mobilalkalmazásban bejelentkezés nélkül hozzá lehet férni a főoldalhoz, a menetrendekhez, a megállókhoz és az élő térképhez. Az egyetlen oldal, ami nem jeleníthető meg azonosítás nélkül az a jegyek oldal, innen a rendszernek át kell irányítani a bejelentkezési felületre. Itt biztosítani kell a regisztráció lehetőségét is, amennyiben a felhasználó még nem rendelkezik fiókkal. A regisztráció során a rendszer le kell ellenőrizzen minden megadott adatot, még az email tulajdonjogát is, azáltal hogy egy ellenőrző kódot küld a megadott email címre, amelyet be kell írni a felületen megjelenő szövegmezőbe. Sikeres regisztráció vagy bejelentkezés után a rendszernek minden adatot biztonságosan el kell mentenie, a jelszavat hash-eltek formában és át kell irányítani a felhasználót a főoldalra.

A főoldalon lévő szövegmezőkben megadott adatok alapján a rendszernek le kell kérnie a szerverről néhány lehetséges járatot, amelyek indulnak az adott településről, adott időben, a megadott célállomásra. A megjelenített találatokhoz tartoznia kell egy jegyvásárlási lehetőségnek is, amelyben meg kell adni a jegyek számát és véglegesíteni kell a vásárlást. A jegyvásárlás is bejelentkezést igényel, így az azonosítatlan felhasználót a rendszer vásárlás előtt átirányítja a bejelentkezéshez. A vásárlás csak a Stripe oldalon történő kifizetés után véglegesíthető, ezt követi a jegy kódjának kigenerálása és elmentése az adatbázisban. Ezek után, a Jegyek oldalon, a felhasználóknak lehetőségük van a jegyek megtekintésére is, amelyeket korábban vásároltak, és amelyek QR kódos formában jelennek meg a buszon való könnyebb beolvasás végett. A menetrendek oldalon a felhasználóknak lehetőségük van a buszok és megállók közötti választásra lenyíló listákból, amelyet követően a rendszer kilistázza az adott járat megállóhoz kötött időpontjait egy táblázatban. Ugyanitt a rendszernek lehetőséget kell biztosítani a buszok útvonalának megjelenítésére térképen. A megálló oldalon a felhasználóknak lehetőségük van az összes megálló megtekintésére térképen, valamint lenyíló listákat használva, a megállók szűrésére település vagy buszjárat alapján is. A térképnek interaktívnak kell lennie, így a megjelölt megállókra való kattintás után meg kell jelennie az adott megálló adatainak egy kis előugró ablakban. Az élő térkép oldalon a felhasználóknak lehetőségük van a buszok valós idejű pozícióinak követésére térképen. A megjelenített buszokat a rendszernek valós időben kell frissítenie, legalább félpercenként, hogy a felhasználók pontos információkat kapjanak a buszok helyzetéről. Az összes térképes oldalon a felhasználóknak lehetőségük van a térkép nagyítására és kicsinyítésére, valamint a térkép mozgatására is, egérrel vagy érintéses gesztusok segítségével. A felhasználónak meg kell tudnia tekinteni a saját pozícióját is a térképeken, amennyiben engedélyezte a GPS-hez való hozzáférést.

Az adminisztrátori weboldalon szükség van azonosításra, anélkül a rendszer minden oldalról átirányít a bejelentkezéshez. Itt a regisztráció nem lehetséges, így külső, admin szerepkörrel nem rendelkező felhasználók nem tudnak hozzáférni az adminisztrátori felülethez. A főoldalon található statisztikák között szerepel az eladott jegyek száma, a megtett kilométerek száma és a bevétel összege, időintervallumok szerint csoportosítva, táblázatos formában. Mivel minden busztársaságnak el kell legyenek különítve az adatai, ezért a szerver mindig a bejelentkezett adminhoz tartozó cég adataiból számolja ki a statisztikákat. A főoldalon a rendszernek meg kell jelenítenie az esetleges értesítéseket is, előugró "Toast" üzenetek segítségével. A megállók oldalán az adminisztrátoroknak lehetőségük van a megállók listázására, az ezekre való egyenkénti kattintás után pedig a szerkesztésükre is. Emellett megállók törlésére és új megállók hozzáadására is lehetőség van egy interaktív térkép segítségével. A térképre kattintva a hely koordinátáit a rendszer beilleszti az űrlapba, ahol meg kell adni még a megálló nevét és a címet is. Utóbbit, amennyiben elérhető, le kell kérnie a rendszernek a térképszolgáltatótól, hogy ezzel is gyorsítsa a létrehozást. A járatok oldalán az adminisztrátoroknak lehetőségük van a buszjáratok kiválasztására egy lenyíló menüből, ezt követően pedig meg kell jelennie a járatszerkesztő űrlapnak. Ugyanitt lehetőség lesz járatok hozzáadására és törlésére is. A menetrendek oldalán az adminisztrátoroknak lehetőségük van a járatok indulási időpontjainak és jegyárainak megtekintésére, szerkesztésére és törlésére. Az órarendek bevitelét le kell lehessen bontani napokra, ezeket pedig dinamikusan megjeleníteni, hogy minél kevesebb helyet foglaljon az oldalon. A buszok és alkalmazottak oldalakon a rendszernek biztosítania kell egy egységes felületet, amelyben lehetőség nyílik az elemek listázására, új elemek hozzáadására, meglévő elemek szerkesztésére és ezek törlésére is. A buszoknál fontos megadni az útdó, biztosítás és műszaki vizsga időpontjait, mert a rendszer ezek alapján fog emlékeztetőket megjeleníteni a felületre való belépések alkalmával. Az alkalmazottaknál meg kell adni a nevet, a beosztást és a telefonszámot, valamint az emailt és a jelszót is, amelyet a rendszer hash-elt formában tárol. Utóbbiak az alkalmazott bejelentkezéséhez lesznek szükségesek, így a rendszernek ellenőriznie kell őket.

### **3.2.2. Nem funkcionális követelmények**

#### **Szervezéssel kapcsolatos követelmények**

- Programozási nyelvek: Dart, HTML, CSS, JavaScript
- Keretrendszerek: Flutter, Vue.js
- IDE: Visual Studio Code
- Verziókövetés: Git és GitHub
- Webhost: Netlify
- Csoportos kommunikáció: Slack

## Termékkel kapcsolatos követelmények

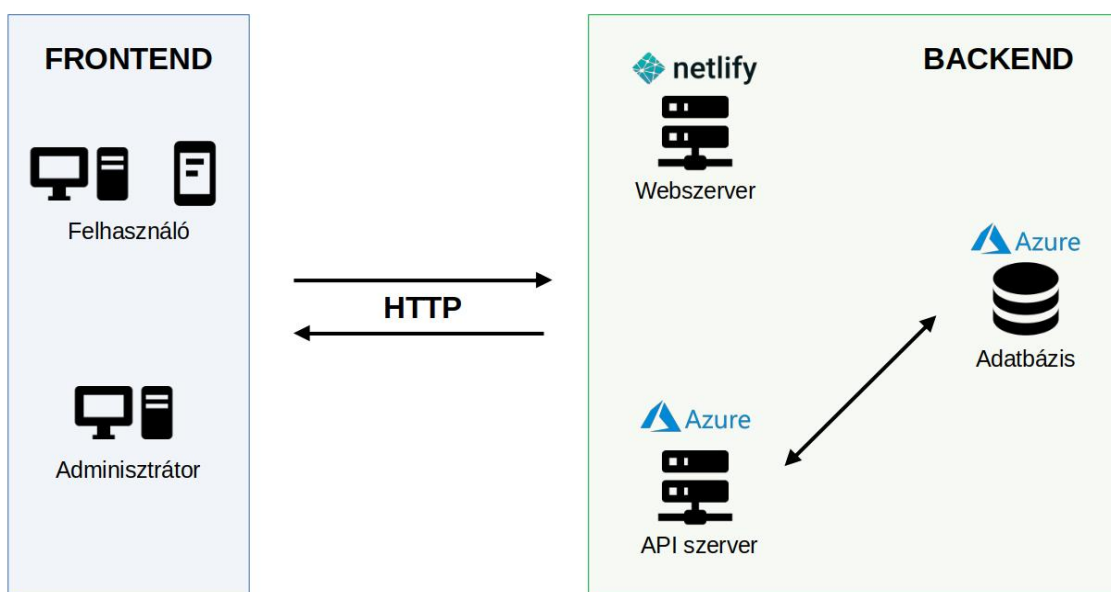
A felhasználói weboldal és az admin felület használatához egy modern, internetezésre alkalmas eszközre van szükség, a térképfunkciókhoz teljes kihasználásához engedélyezni kell a helymeghatározást. Ajánlott a 2018 után kiadott böngészőverziók használata:

- Chrome  $\geq 87$
- Firefox  $\geq 78$
- Safari  $\geq 14$
- Edge  $\geq 88$

Az mobilapplikáció használatához szükség van egy okostelefonra vagy tabletre, amelyen minimum Android 5.0 vagy iOS 8.0 operációs rendszer fut. Fontos hogy legyen rajta internet elérés (3G,4G,5G vagy Wi-Fi), legalább 120MB szabad tárhely és 516-1024 MB szabad memória. Itt is ajánlott a helymeghatározás (GPS) engedélyezése, hogy a térképfunkciók helyesen működjenek.

## 3.3. A rendszer architektúrája

Az általunk tervezett rendszer architektúra diagramja a 3.2 ábrán látható. A rendszer frontend és backend része jól elkülöníthető egymástól, az egyes komponensek közötti kommunikáció titkosított HTTP kérések segítségével valósul meg. A továbbiakban külön fogom tárgyalni a frontend és backend részeket.



3.2. ábra. A rendszer architektúrája

### 3.3.1. Frontend

A frontend a felhasználói felületet biztosítja, amelyen keresztül az emberek interakcióba léphetek a rendszerrel. Jelen esetben a frontend két weboldalból és egy mobilalkalmazásból áll. A két weboldal teljesen elkülönül egymástól, mivel az egyik az adminisztrátoroknak, míg a másik a végfelhasználóknak készült. A weboldalak működése eltér a klasszikustól, mivel kliens oldali renderelést használnak, azaz a felhasználói felületet a kliens oldalon generálja a böngésző, nem pedig a webszerverről érkezik készen. Jelen esetben a webszerver csak a statikus fájlokat szolgáltatja, a dinamikus tartalmakat a kliens oldali JavaScript illeszti be az oldalakba. Ebből a szempontból a weboldal és a mobilalkalmazás működése megegyezik, mivel mindkettő direkt módon az API szerverrel kommunikál, REST kéréseken keresztül. Az adatok JSON típusú szöveggént közlekednek a végpontok között az egyszerű feldolgozhatóság végett. Fontos megemlíteni, hogy egyes API végpontokhoz azonosítás szükséges, ami úgynevezett tokonek segítségével valósul meg. Úgy a weboldalak, mint az alkalmazás eltárolják a szervertől kapott tokenet és hozzáfűzik minden API kéréshez, ezzel azonosítva a felhasználót. Egyes oldalakon térképek is megjelennek, amelyeket a web esetében a MapBox, míg a mobil esetében a Google Maps szolgáltat. Mindkét esetben az adott platformhoz és technológiához közelebb álló térképszolgáltatót választottam, hogy a legjobb felhasználói élményt biztosítsam.

### 3.3.2. Backend

A backend a szerveroldali logikát és az adatbázis elérést biztosítja. Ahogyan az előbbieken említettem, a weboldalak API hívásokon keresztül kapják meg az adatokat és dinamikusan jelenítik meg őket, ezért nincs egy olyan webszerver, amely backend oldalon renderelné ki a weboldalakokat, csak egy olyan, amely statikus fájlokat szolgáltat. Ez azt eredményezi, hogy a webszerver nem tartalmaz üzleti logikát és az adatbázissal sincs összeköttetésben. Ezzel szemben van egy API szerver, amely az összes felhasználói felületet kiszolgálja adatokkal. Az ő feladata az adatbázissal való kommunikáció, az adatok feldolgozása és az ezek elérhetővé tétele a frontend számára, API végpontok formájában. Ez a komponens kezeli a felhasználóktól érkező kéréseket is, amelyek az adatbázist szeretnék módosítani. Minden ellenőrzés, biztonsági vizsgálat is itt folyik le, hogy a frontend csak a megfelelő adatokhoz férhessen hozzá és csak azokat módosíthassa. Ezekben a folyamatokban az adatbázisnak csak az a feladata, hogy a felhasználók, buszok, járatok, megállók és jegyek adatait biztonságosan tárolja és azokat az API szerver számára elérhetővé tegye olvasásra és módosításra is.

## 4. fejezet

# A felhasználói felület tervezése

Az egyik legfontosabb része a projektnek a UI, mert ezt a részét látják a felhasználók és ezzel fognak interakcióba lépni. Emiatt kiemelkedően fontos, hogy az jól meg legyen tervezve. Szerencsére ennek a feladatnak a megkönnyítésére léteznek jól bevált módszerek, amelyek nagyban felgyorsítják és megkönnyítik a munkát. Az én választásom a Figma tervezőprogramra és a Material Design 3 irányelveire esett, amelyeket a következőkben fogok bemutatni.

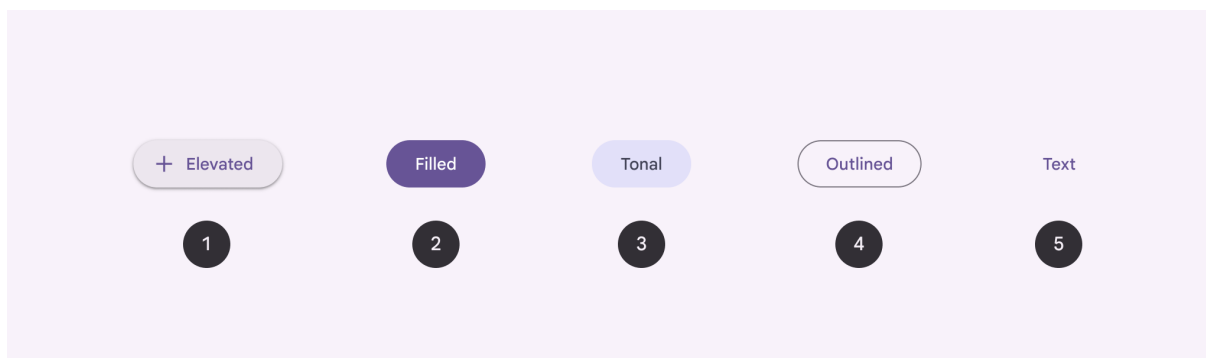
### 4.1. Material Design

Mivel nehéz egy olyan felhasználói felületi stílust kitalálni, ami tökéletesen működik, egységes és jól is néz ki, ezért a Google Material Design irányelveit követve terveztem meg a felületeket. A Material Design egy olyan nyílt forráskódú dizájnnyelv, amelyet követve egységes, jól kinéző és könnyen használható felületeket lehet készíteni. Ez tartalmazza mindazokat a felületi elemeket, amelyekre szükség lehet egy modern weboldal vagy mobilalkalmazás elkészítésekor, mint például a gombok, ikonok, szövegek, színek, betűtípusok, stb. Mindenről precíz leírás van a hivatalos dokumentációban, így könnyű követni az irányelveket és a felhasználói felület egységességét is biztosítani. Minden komponenshez elérhető egy előre elkészített Figma elem, így ezeket csak be kell húzni a tervezőfelületre, nem kell kézzel létrehozni. Emellett az elemekhez tartozik implementáció is, amely elérhető Flutterben, Androidban és weben is, így a fejlesztés során nem kell lekódolni őket, csak be kell importálni a projektbe. Ezáltal a fejlesztés gyorsabb és egyszerűbb lesz, és a felhasználói felület egységessége is garantált <sup>1</sup>.

Én a legújabb, 3-as verzióját használom a Material Design-nak, amely a legmodernebb és a legnagyobb támogatással rendelkezik. Ez egy jól bevált UI-t eredményez, amelyet már sok alkalmazás használ, köztük a Google összes alkalmazása is, így már szinte mindenki találkozhatott vele. Egy példa az MD3-as gomb stílusokra a 4.1 ábrán látható. Ilyen és ehhez hasonló komponensekből épül fel az általam tervezett UI is.

<sup>1</sup> Material Design 3: <https://m3.material.io/>





4.1. ábra. A Material Design 3 gomb stílusai <sup>2</sup>

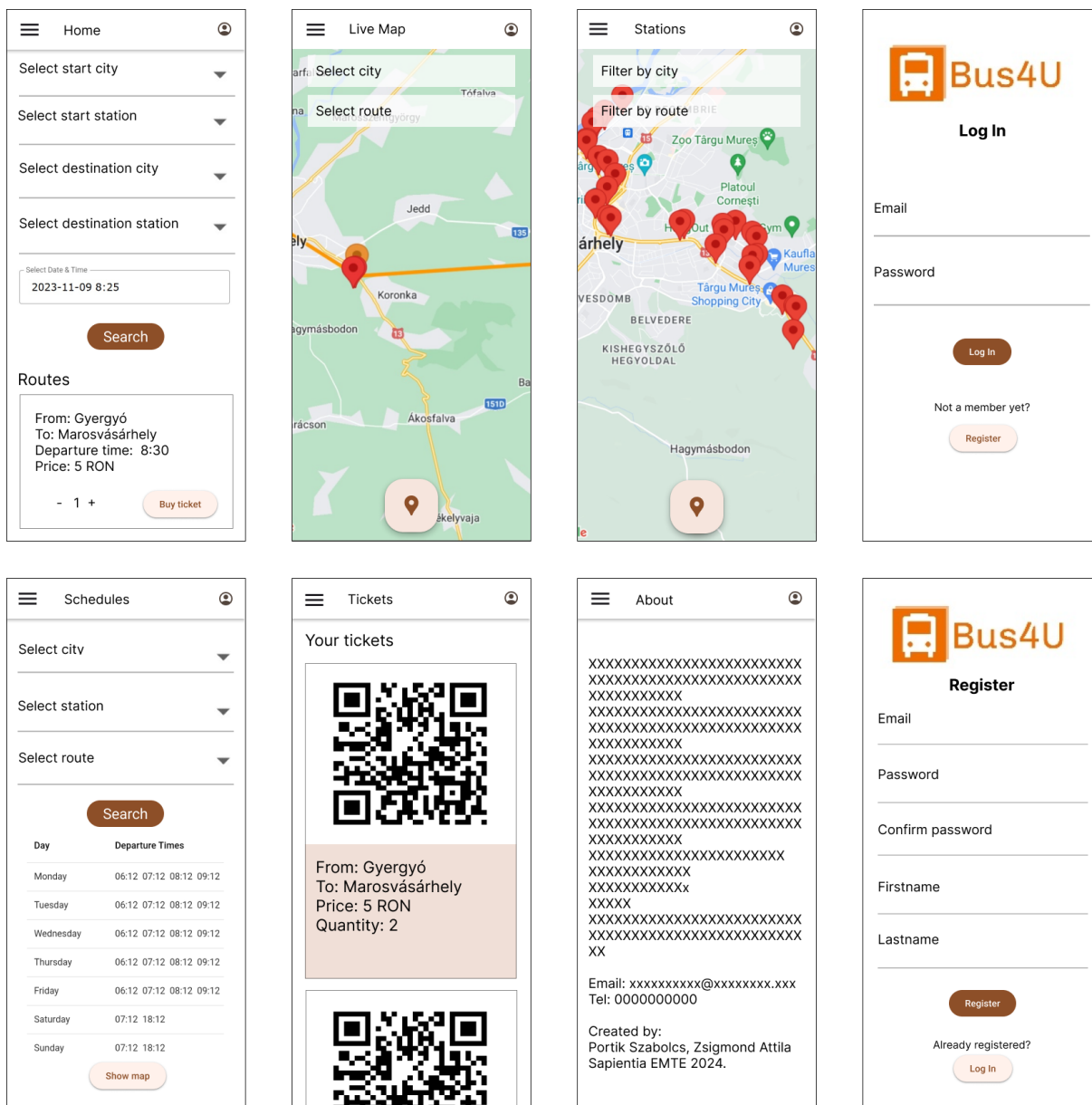
## 4.2. Figma terv

A UI terv vizuális elkészítésére a Figma segédprogram a legmegfelelőbb választás rengeteg szempontból. A Figma egy online és offline egyaránt használható felülettervező program, amely lehetővé teszi több felhasználó együttműködését is, például a dizájnér és a fejlesztő közös munkáját. A Figmában egyszerűen, fogd és vidd módszerrel lehet hozzáadni elemeket a munkavá- szonhoz, de aki pontosabban szeretne tervezni, annak az oldalsávban van lehetősége pixel pon- tossággal megadni az elemek tulajdonságait is. Mint említettem, a Figmában elérhetőek előre elkészített Material Design 3-as felületi elemek is, amelyek használata sokszorosan lerövidí- ti a tervezéshez szükséges időt. A felületi elemekhez funkcionalitást is lehet rendelni, például ha egy gombra kattint a felhasználó, akkor egy másik oldalra navigáljon. A vizuális tervezés mellett lehetőség van a UI terv programozására is, de akár a fejlesztéshez szükséges kódot is kigenerálhatjuk az elkészített tervekből. A Figma egy nagyon hasznos eszköz, amely segítsé- gével gyorsan és egyszerűen lehet elkészíteni egy felhasználói felületet, amelyet a fejlesztők is tudnak használni a fejlesztés során <sup>3</sup>.

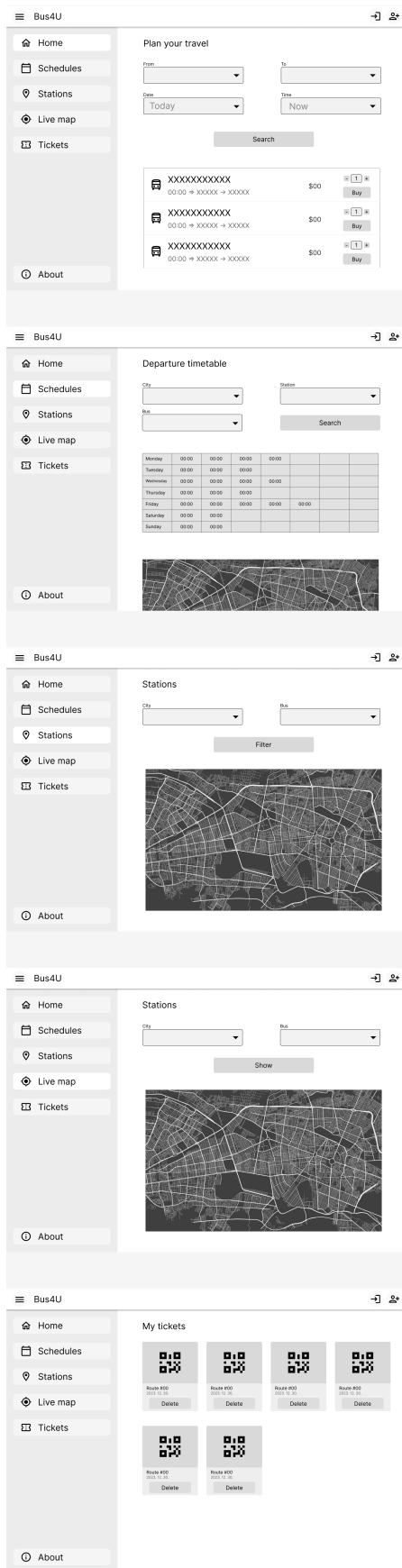
Jelen esetben én vagyok a tervező és a fejlesztő is egyben, így az én feladatom a UI terv elké- szítése, de a Figma-t használva, tervezői szaktudás nélkül is sikerült megoldani a feladatot. Az itt létrejött wireframe-eket (drótvázakat) követve már gyorsabb volt a fejlesztés. Munka közben jöttek további ötletek is, így a végső termék nem teljesen egyezik a tervvel, de iránymutatásnak megfelelt. A Material Design irányelvei alapján Figma-ban elkészített mobilalkalmazás terve a 4.2 ábrán látható, míg a felhasználói weboldal terve a 4.3 ábrán, az adminfelület terve pedig a 4.4 ábrán látható.

<sup>2</sup> Forrás: <https://m3.material.io/components/buttons>

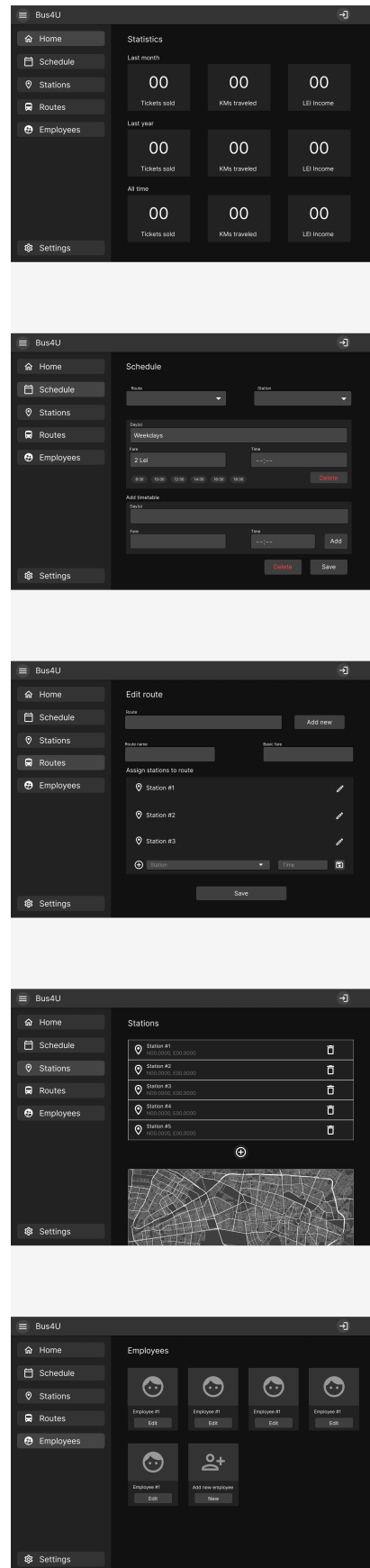
<sup>3</sup> Figma: <https://www.figma.com>



4.2. ábra. A mobilalkalmazás terve



4.3. ábra. A felhasználói weboldal terve



4.4. ábra. Az admin weboldal terve

## 5. fejezet

# Gyakorlati megvalósítás

### 5.1. Felhasznált technológiák

A felhasználói és az admin weboldalak Vue.js keretrendszerben készültek, Vuetify komponenteket felhasználva. Ennek a két technológiának a kombinációja modern, esztétikus és gyors weboldalakat eredményezett, amelyeknek a fejlesztése is egyszerű volt. Az mobil platformokra szánt alkalmazás Flutter technológiával készült, amely által egyetlen kódbázisból ki lehetett generálni Android és iOS készülékekre is a natív alkalmazást. Mindkét implementáció a Material Design irányelveit követi, így a felhasználók egy egységes felületet láthatnak, bármilyen módon és eszközön használják a Bus4U-t.

#### 5.1.1. Vue.js

A Vue.js egy progresszív JavaScript keretrendszer, amelyet azért hoztak létre, hogy a webes felhasználói felületek fejlesztése egyszerűbb és gyorsabb legyen. A JavaScript keretrendszerek célja, hogy megkönnyítsék a webprogramozást, azáltal hogy előre megírt osztály- és függvénykönyvtárakat szolgáltatnak, amelyek segítenek lerövidíteni a fejlesztési időt. Első ilyen keretrendszer a jQuery volt, amely a DOM (Document Object Model) manipulációt és az AJAX (Asynchronous JavaScript And XML) hívásokat tette egyszerűbbé, majd később a React és az Angular keretrendszerek jöttek, amelyek a komponens alapú fejlesztést tették lehetővé. Ez a fejlődés viszont mind nagyobb és nagyobb kiterjedésű csomagokat eredményezett, ami kissé csökkentette a weboldalak teljesítményét. A Vue.js ezt a problémát próbálja megoldani, azzal hogy egy könnyű, gyors és hatékony keretrendszert kínál, amely egy Virtuális DOM modell segítségével gyorsítja a futást és amelynek a forráskódja akár 24Kb-ra is zsugorítható, így viszonylag gyors a betöltése is [3].

A Vue.js két alapköve a deklaratív UI felépítés és a reaktivitás. A deklaratív felépítés azt jelenti hogy az alap HTML kódot a Vue kiegészíti egy saját sablon-szintaxissal, amely lehetővé teszi a változók beágyazását a HTML kódba. A reaktivitás azt jelenti, hogy amikor a HTML

sablonban felhasznált JavaScript változók értéke megváltozik, akkor a Vue érzékeli ezt és automatikusan megváltoztatja a megjelenített HTML kódot, így mindig a legfrissebb adatokat látja a felhasználó. Ugyanez végbemegy fordított irányban is, mert a felhasználói interakciók eredményeképpen a Vue automatikusan frissíti a háttérben lévő változókat is, így lesz a kommunikáció a DOM és a Vue között kétirányú [3, 2].

A Vue keretrendszer egyik legnagyobb előnye, hogy könnyen tanulható és alkalmazható, így gyorsan lehet vele fejleszteni. Ennek egyik oka, hogy komponens alapú, amely lehetővé teszi a weboldalak felépítését újrafelhasználható, különálló komponensekből, amelyeket később össze lehet fűzni egy nagyobb alkalmazássá. Ezek úgynevezett Single-File Component-ek (egyfájlos-komponensek) formájában jelennek meg, amely azt jelenti, hogy az adott komponens HTML, CSS és JavaScript kódja mind egy fájlban belül található, ezzel egy átlátható és moduláris alkalmazásfelépítést eredményezve [2]. Emellett a Vue nagyon jó online dokumentációval rendelkezik, amelyben minden funkció részletesen le van írva, de különleges problémákkal és kérdésekkel a dinamikusan növekvő fejlesztői közösséghez is lehet fordulni, így mindig könnyen meg lehet találni az éppen szükséges információt és nagyon gyorsan el lehet sajátítani a keretrendszer használatát [3].

A Vue legújabb verziója a Vue 3, amely a komponensek leírására 2 féle API-t (Application Programming Interface) is biztosít: Options és Composition. A Composition API a 3-as verziótól került bevezetésre és különféle teljesítménybeli optimalizációkat tartalmaz a régi Options API-jal szemben, de a szintaxisa is egyszerűbb, mert közelebb áll az alap JavaScript szintaxisához [2]. A Vue 3 a TypeScript támogatását is bevezette, amely egy statikus típusellenőrző nyelv a JavaScript fejlesztéséhez és amely segít a hibák korai felfedezésében, valamint a kód minőségének javításában. Én a munkám során a Vue 3-at és a Composition API-t használtam, mivel ezek a legújabb és legelőnyösebb technológiák jelenleg.

Fontos megemlíteni a Vue ökoszisztémájába tartozó egyéb eszközöket is, amelyek segítik a fejlesztést. Ilyen például a Vue Router, amely a weboldalak közötti navigációt teszi lehetővé, a Pinia, amely a globális állapotkezelést segíti, a Vite, amely a projekt generálását és a fejlesztést segíti, vagy a Vitest amely a teszteléshez szolgáltató hasznos eszközöket [2]. Ezek közül kiemelném a Vite nevű build tool-t (építő eszközt), amelynek a gyors újratöltés funkciója figyeli a forráskódban létrejött módosításokat és azonnal frissíti a weboldalt, így a fejlesztőnek nem kell manuálisan a frissítés gombot nyomogatni minden begépett kódsor után.

A felhasználó szempontjából a leglátványosabb tulajdonság, hogy nagyon gyors a betöltés és a navigáció, mivel a Vue Single Page Application (Egyoldalas Alkalmazás) üzemmódja és a Vue Router kombinációja által lényegében csak egyszer töltődik be a webalkalmazás, utána az oldalak közötti navigáció során csak a tartalom frissül, nem pedig az egész oldal, így lényegesen kevesebb adat közlekedik a hálózaton keresztül és a böngészőnek is kevesebb dolga van az oldalak megjelenítésénél [2]. Ezeknek a funkcióknak a segítségével egy gyors és hatékony felhasználói felületet lehet készíteni, amely a gyengébb hálózati lefedettséggel és a gyengébb hardverrel rendelkező eszközökön is jól használható.

### 5.1.2. Vuetify

A Vuetify egy Material Design alapú Vue.js-re épülő, nyílt forráskódú UI könyvtár, amely lehetővé teszi a gyors és egyszerű felhasználói felületek készítését, azáltal hogy rengeteg előre elkészített Vue komponenst tartalmaz, amelyeket egyszerűen be lehet integrálni a weboldalba, így sok időt lehet vele spórolni. A Vuetify segítségével gyorsan és hatékonyan sikerült felépítenem a weboldalak felhasználói felületeit, mi több, ezek jól néznek ki és jól is működnek minden tesztelt eszközön.

### 5.1.3. Flutter

A Flutter egy nyílt forráskódú, Google által kifejlesztett, cross-platform, UI készítő keretrendszer, amely lényege, hogy egyetlen kódbázisból ki lehet generálni az alkalmazásokat a legnépszerűbb platformokra. Ezek a platformok a következők: Android, iOS, Windows, macOS, Linux, valamint a web.

A Flutter a Dart programozási nyelvet használja, amely szintén a Google fejlesztése. Ez egy modern, típusos, objektum orientált nyelv, amelyet könnyen és gyorsan el lehet sajátítani. Eredetileg a Dart a JavaScript lecserélésére volt létrehozva a webfejlesztésben, ami eddig nem tudott valósulni, viszont idővel a Flutter technológia alapjává vált és így tett szert népszerűsége. A Flutter egyik nagy előnye, hogy egy widget rendszert használ, amely lehetővé teszi a felhasználói felületek egyszerű és gyors elkészítését, valamint a különböző platformokra történő kiadást. Ezek a widgetek alapértelmezetten a Material Designra épülnek, így egy egységes, intuitív és a felhasználót megragadó felület építhető fel belőlük [1], amely összhangban van a Vuetify webes komponenseivel is. A Flutter egy másik nagy előnye, hogy a natív alkalmazásokhoz hasonlóan gyors és hatékony, mivel natív kódra fordítja le a Dart kódot, így a lehető legjobb teljesítményt nyújtja, bármilyen platformról is legyen szó [6].

A funkció amely nagyban megkönnyíti a fejlesztést és a tesztelést, az a Hot Reload, amely lehetővé teszi a fejlesztők számára, hogy azonnal láthassák a változtatásokat a kódban, anélkül hogy újra kellene indítani az alkalmazást [6]. Ez a Flutter funkció sokáig egyedüli volt a mobilfejlesztésben, az Android és az iOS csak később vezetett be hasonló funkciókat.

A Flutterben való alkotás hatékonyságát tovább növeli az, hogy egy részletes online dokumentációval rendelkezik, és hogy a Pub Package Manager-t (Csomagkezelőt) használja, amely rengeteg előre elkészített csomagot tartalmaz, az egyszerű UI komponensektől kezdve, a komplex háttérfolyamatokig. Ehhez csak meg kell keresni a megfelelőt a `pub.dev` weboldalon és be kell illeszteni az alkalmazásba. [6].

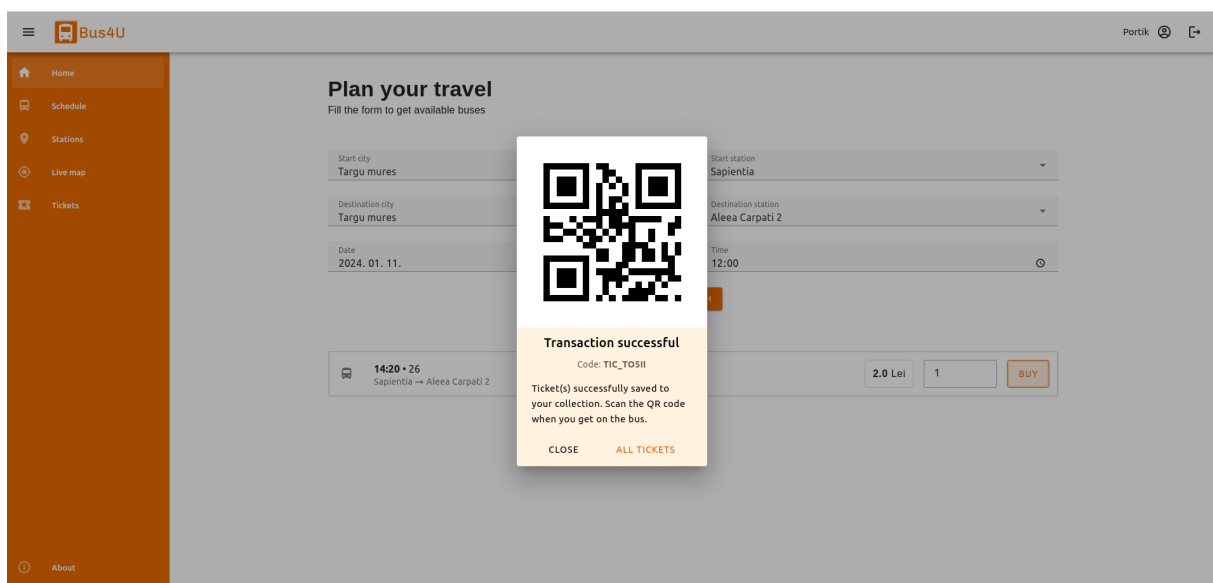
Jelen projekt esetében, mivel a webes felületet a Vue.js és a Vuetify keretrendszerekkel készítettem, a Flutter használatakor inkább a mobilos platformokra koncentráltam, tehát az Androidra és az iOS-re. Összességében így kijelenthető, hogy a legnépszerűbb platformok le lettek fedve és bárki, bármilyen eszközről el tudja érni a Bus4U-t.

## 5.2. Felhasználói weboldal és alkalmazás

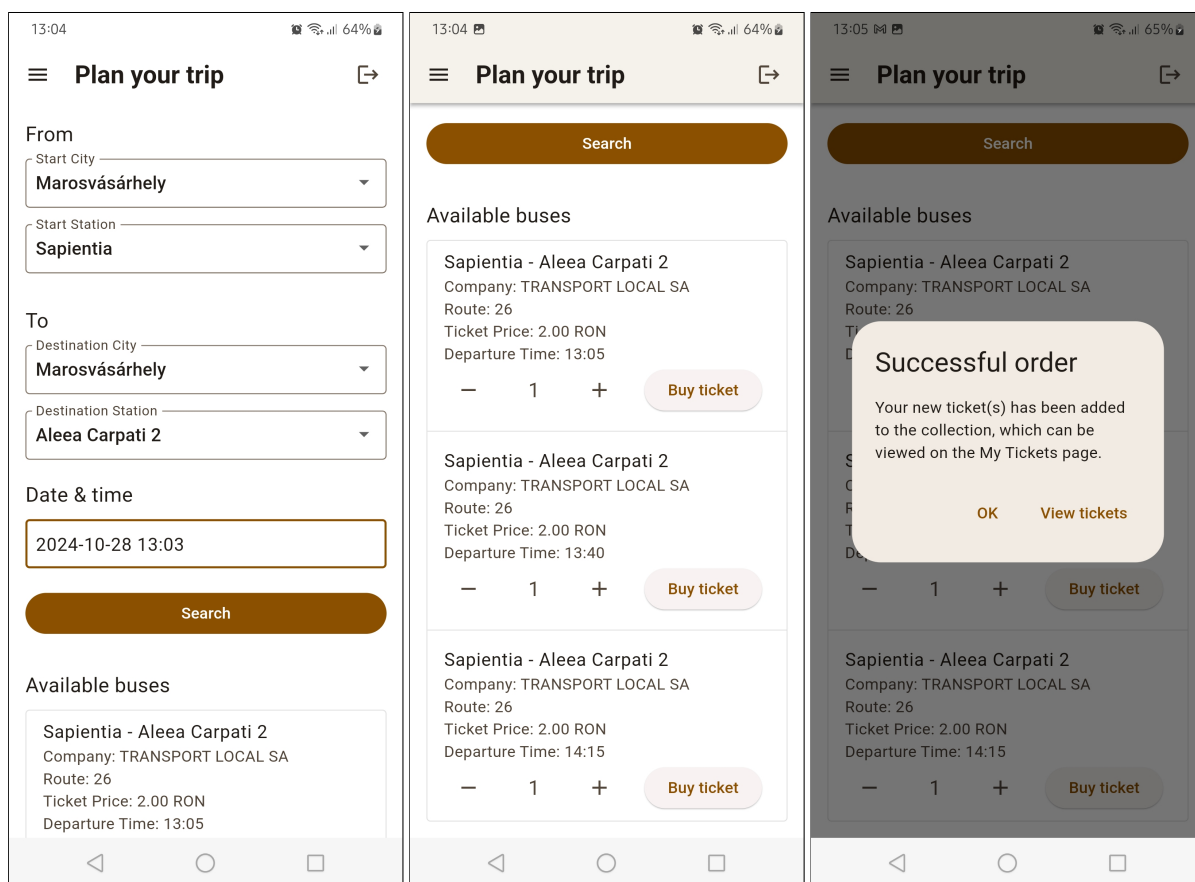
A felhasználói felület működése megegyezik a weboldalon és a mobilalkalmazásban is, azonban a megjelenésük kicsit eltér egymástól, mivel a weboldal a böngészőben fut, míg az alkalmazás a mobil eszközön. Ettől függetlenül a weboldal is tökéletesen használható az okostelefonokon, mivel reszponzívra terveztem és alkalmazkodik bármilyen kijelzőmérethez, így aki nem szeretne alkalmazást telepíteni, az is könnyen hozzáférhet a Bus4U-hoz böngészőből. A felületek létrehozásakor törekedtem a hasonlóságra, így a navigációs menü, a UI komponensek és a szín-paletta is megegyezik, így a felhasználó könnyen átláthatja és használhatja mindkét platformot.

### 5.2.1. Főoldal

A weboldalra rákeresve a felhasználót a kezdőoldal fogadja, ahol lehetőség van egy utazás megtervezésére. Ehhez ki kell választani a kiindulási helyet és megállót, majd a cél helyét és a megállóját, végül pedig az utazás dátumát és időpontját. Ekkor a rendszer lekéri a szerverről a megadott megállók közötti, az adott napon és a megadott időpont után közlekedő járatokat, valamint azok jegyárát. Minden egyes megjelenő járatnál ki lehet választani a jegyek számát, amely alapértelmezetten egy jegyre van beállítva. Ezt követően a felhasználó, a vásárlás gombra kattintva megszerezheti a jegyet vagy jegyeit az adott buszjáratra, amennyiben be van jelentkezve, ellenkező esetben a rendszer átirányítja a bejelentkezéshez.



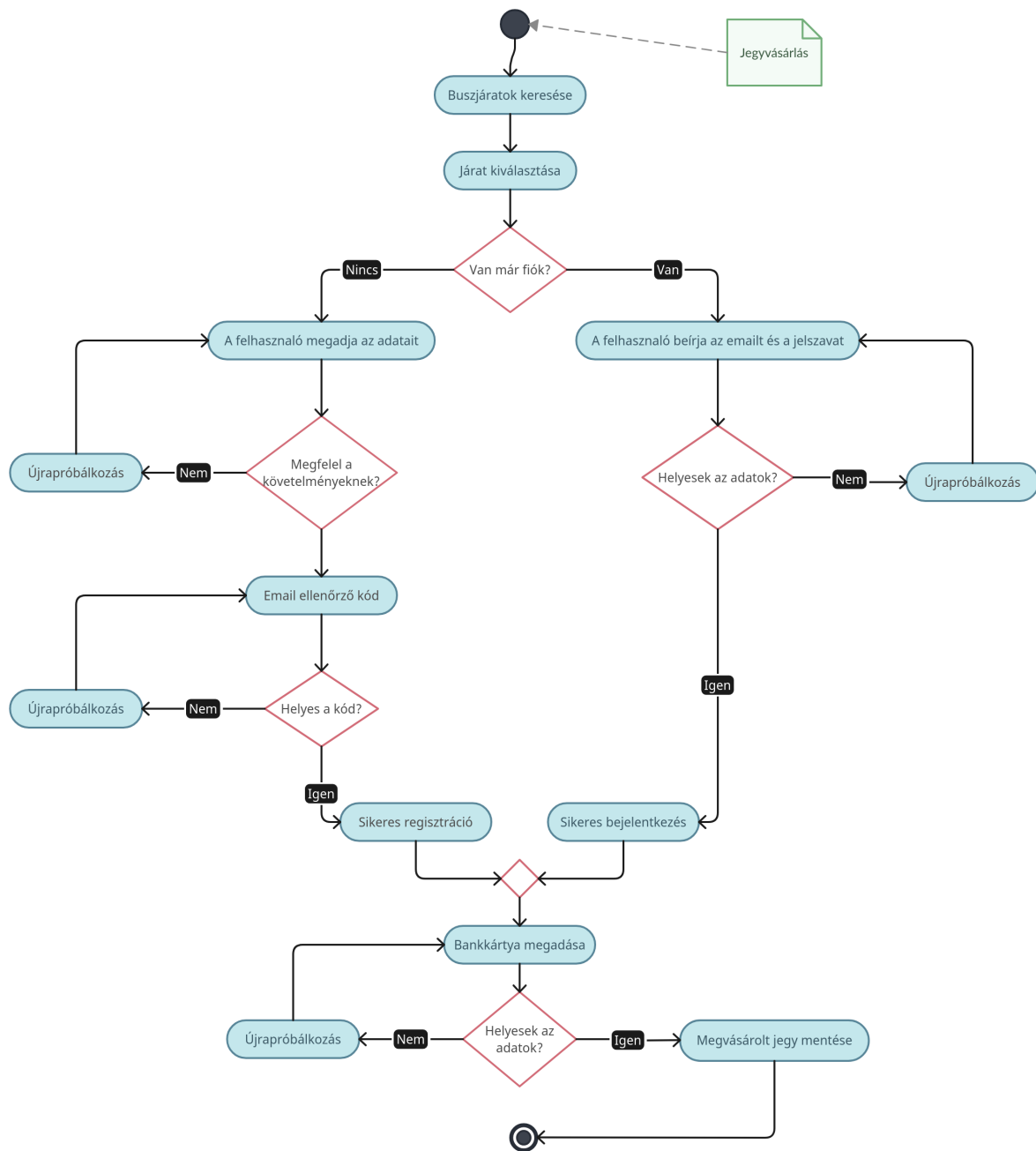
5.1. ábra. Jegyvásárlás a weboldalon



5.2. ábra. Jegyvásárlás a mobilalkalmazásban



A könnyebb megértés végett, a jegyvásárlási és bejelentkezési folyamat szemléltetésére elkészítettem a 5.3 ábrán lévő aktivitás diagramot:



5.3. ábra. A jegyvásárlás folyamata

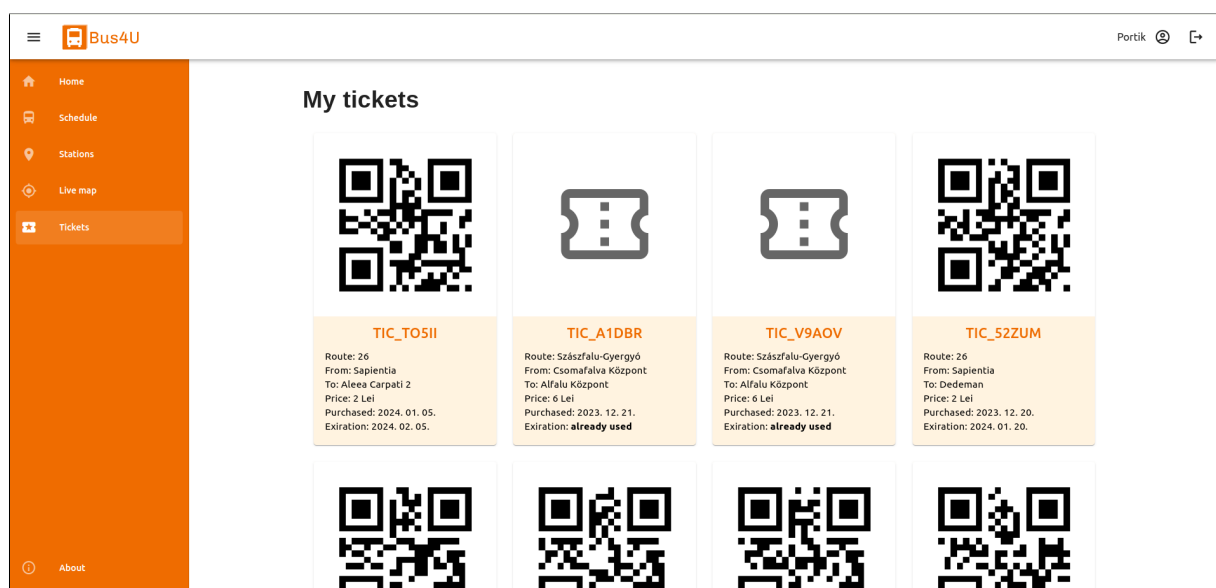
Ahogy az ábrán is látható, miután a felhasználó megtalálta a megfelelő buszjáratot és rákattintott a jegy(ek) megvásárlására, a rendszer átirányítja őt a bejelentkezéshez, itt, ha van már fiókja, bejelentkezhet a meglévő adataival, amennyiben nincs, úgy átléphet a regisztrációs oldalra, ahol meg kell adnia a nevét, email címét, és egy új jelszót. A rendszer minden adatot ellenőriz és ha valamelyik hibás (pl. érvénytelen email cím vagy túl rövid jelszó) akkor egy ennek megfelelő hibüzenetet ír ki. Ez addig ismétlődik, amíg minden adat megfelelő lesz, aztán a szerver

automatikusan küld egy 4 számjegyből álló ellenőrző kódot a megadott e-mail címre, amelyet a felhasználó be kell írjon az alkalmazásban megjelenő szövegmezőbe. Ha ez a kód helytelen, akkor a folyamat kezdődik előlről, a felhasználónak új adatokat kell megadnia. Ha regisztráció sikeres és az ellenőrzőkód helyes, akkor a rendszer átirányítja a felhasználót a bankkártyás fizetési oldalra, ami jelenleg nincs implementálva, ez a jövőbeli tervek között szerepel. Ezután már csak az maradt, hogy a backend generál egy egyedi kódot a megvásárolt jegyhez és megjelenik a kijelzőn a sikeres vásárlás visszaigazolása. Az itt megvásárolt jegye(ke)t a felhasználó a Jegyek nevű oldalon találja meg szöveges és QR-kódos formátumokban.

## 5.2.2. Jegyek

A megvásárolt jegyeket a rendszer elmenti a felhasználó gyűjteményébe, amely a Jegyek oldalon található, így ezek bármikor visszakereshetőek és felhasználhatóak lesznek, kivéve ha lejár az egy hónapos érvényesség. A jegyen megjelenített adatok között van a járat neve, a kiinduló- és célállomás, az ár, valamint a vásárlás és az érvényesség dátumai. A már elhasznált jegyek megvannak jelölve és el van rejtve a rajtuk lévő QR-kód így azokat nem lehet többször felhasználni. A jegyek a létrehozási idejük szerinti csökkenő sorrendben vannak listázva, így a legfrissebb jegyek mindig az elején vannak.

A itt megjelenő jegyeket úgy lehet felhasználni, hogy a buszra való felszálláskor a sofőr mellett elhelyezett POS terminálra kell mutatni az aktuális jegy QR-kódját, az pedig leolvassa és elküldi a szervernek ellenőrzésre. Ha a kód érvényes és minden rendben van, akkor a felhasználó felszállhat a buszra, a jegy pedig invalidálódik.



5.4. ábra. A felhasználó megvásárolt jegyei

### 5.2.3. Menetrend

A következő oldal a Menetrend, amely hasonlóan az egyes buszmegállókban kihelyezett táblázatokhoz, megjeleníti a kiválasztott buszjárat indulási időpontjait, az adott megállóból, napokra lebontva. A mi megoldásunk viszont tartalmaz emellett egy interaktív térképet is, ahol a felhasználó meg tudja tekinteni a kiválasztott járat útvonalát, az érintett megállókat és az útvonalon közlekedő buszok élő helyzeteit is, a saját pozíciója mellett.

Bus4U

Portik

## Schedule

Select a station and a bus to view the departure timetable and the route marked on the map

City  
Marosvásárhely

Station  
Dedeman

Bus  
Gyergyó-Marosvásárhely

SEARCH

Departure times for station: Dedeman and bus: Gyergyó-Marosvásárhely

|           |       |       |
|-----------|-------|-------|
| Monday    | 07:45 | 19:45 |
| Tuesday   | 07:45 | 19:45 |
| Wednesday | 07:45 | 19:45 |
| Thursday  | 07:45 | 19:45 |
| Friday    | 07:45 | 19:45 |

5.5. ábra. Listázott menetrend és az útvonal térképe

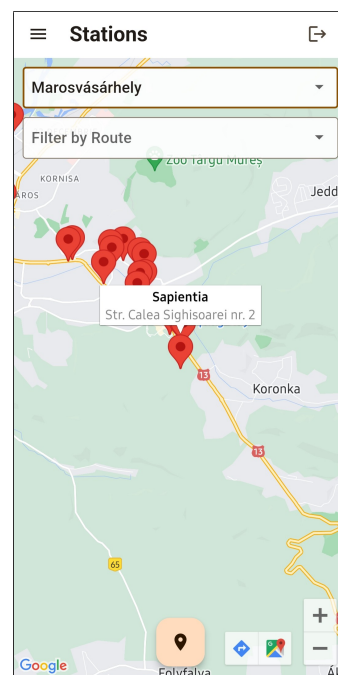
25

## 5.2.4. Megállók és élő térkép

Ez a két oldal az előbb említett térképes funkciókat mutatják meg külön-külön, egyszerűbben. A Megállók oldal megjeleníti a térképen az összes megállót, és ezek között szűrést is biztosít város vagy buszjárat szerint. Az egyes megállókra kattintva a térképen megjelenik egy kis buborék a kiválasztott megálló adataival, mint a név és a cím, így könnyebb megtalálni a keresett megállót.

Az élő térkép oldalon pedig egy-egy útvonalon közlekedő buszok élő pozícióját lehet követni, miután kiválasztottuk a várost és az útvonalat. Ezeket az adatokat a buszokon lévő POS terminálok szolgáltatják, így biztosak lehetünk abban, hogy a buszok helyzetei valós időben jelennek meg. Ez a funkció abban segít, hogy ne maradjunk le, ha egy busz siet, illetve ne várakozzunk feleslegesen a megállóban, ha egy busz késik, így időt lehet vele spórolni.

Mindkét oldalon megjelenik a felhasználó jelenlegi pozíciója, amennyiben engedélyezte az ehhez való hozzáférést a telefonján, így könnyen megtalálhatja a legközelebbi megállót vagy buszt.



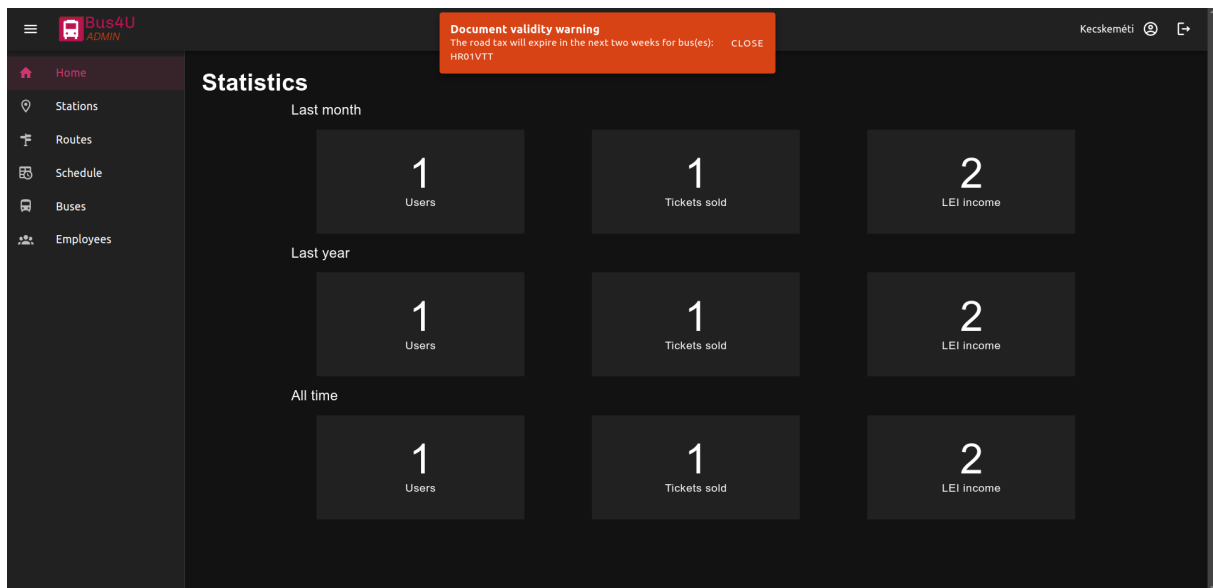
5.6. ábra. Megállók

## 5.3. Adminisztrátori weboldal

Az adminisztrátori felület csak bejelentkezéssel elérhető, ehhez pedig egy admin típusú felhasználói fiók szükséges, amely a kezelendő céghez van hozzárendelve. A cégek adatai el vannak különítve, így minden adminisztrátor csak a saját cégéhez tartozó adatokat láthatja és módosíthatja. Egy cégnek végtelen számú admin fiókja lehet, viszont ezek között lehetnek különbségek, amelyet a szerepkörök határoznak meg. Egyes adminok csak bizonyos funkciókat érhetnek el, míg mások mindenhez hozzáférhetnek. A sofőr szerepkörben lévő felhasználó például csak a buszok adatait láthatja és kezelheti, míg a menedzser szerepkörű adminisztrátor minden egyéb funkcióhoz is hozzáfér.

### 5.3.1. Kezdőoldal/Statisztikák

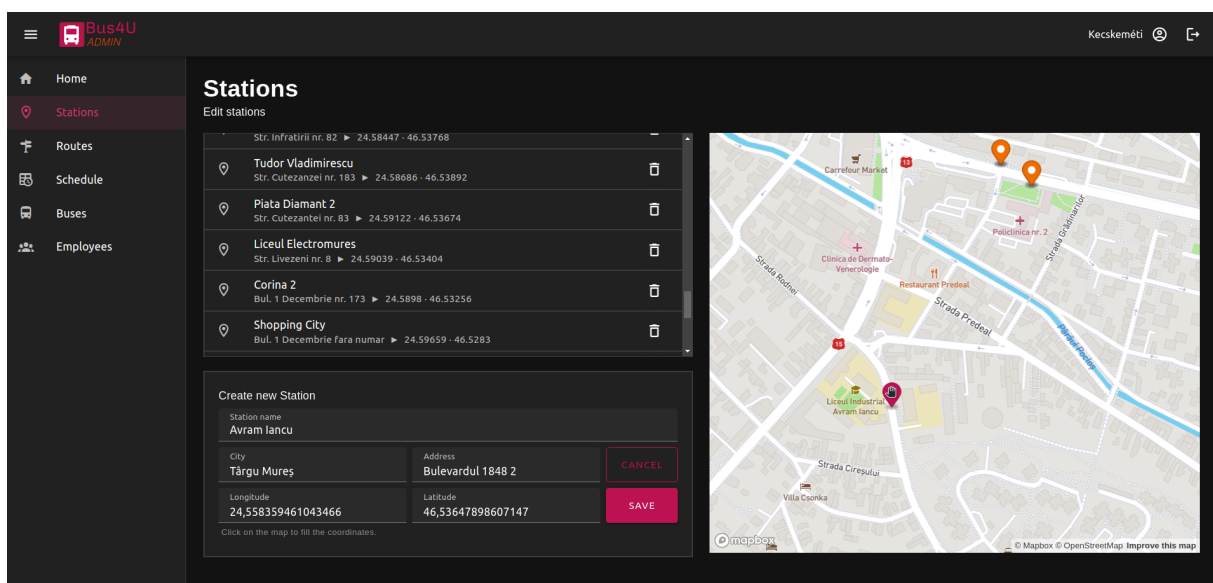
Sikeres bejelentkezés után egy áttekintő oldal jelenik meg, amelyen látható egy kis statisztika a regisztrált felhasználók számával, a megvásárolt jegyek számával és a bevétellel, mindezek havi, évi és mindenkori bontásban. Ez a statisztika segíthet a cégvezetőknek a tanulságok levonásában és a fontos döntések meghozatalában is. Ezen kívül itt jelennek meg az esetleges figyelmeztetések és értesítések is, mint például a buszok lejárt útdíja vagy a közelgő műszaki vizsgálat, így biztosan nem felejtődnek el a karbantartással kapcsolatos események.



5.7. ábra. Kezdőoldal statisztikával és értesítéssel

### 5.3.2. Megállók

A Megállók oldalon egy lista fogad a meglévő megállókkal és azok adataival, mint a név, a hely és a földrajzi koordináták. Egy megállóra rákattintva ennek pontos helyzete megjelenik a lista mellett található interaktív térképen. A listaelemeken található egy törlés gomb is amellyel megszüntethetjük az adott megállót. A lista alatt létre lehet hozni új megállókat is, ehhez csak meg kell adni a megálló nevét, a várost, a címet és a koordinátákat. A rendszer képes automatikusan is kitölteni a néven kívül az összes adatot, ha a felhasználó a megálló tervezett helyére kattint a térképen, ahogy az 5.8 ábrán is látszik.



5.8. ábra. Adminisztrátori megállók oldal

### 5.3.3. Útvonalak

Az Útvonalak oldalon lehet szerkeszteni a buszjáratok útvonalait vagy újakat összerakni a létező megállókból és itt lehet megszabni a járatok alap jegyárát is. Először ki kell választani egy lenyíló listából a szerkeszteni kívánt útvonalat vagy a mellette lévő gombbal újat lehet hozzáadni. Ezután megjelenik az útvonal-szerkesztő, ahol meg lehet adni a nevet, az alap jegyárát és egy lenyíló listából egyesével ki lehet választani az előző oldalon létrehozott megállókat. A járatok alap jegyára kezdésből 0.0 értékű, mert ez csak olyan esetekben használatos, ahol nem a megállók közötti távolság alapján számolják a jegyárát, hanem fix áron. Ilyen esetben ez az ár minden jegyre érvényes, indulási és érkezési helytől függetlenül. Alább megjelenik a hozzáadott megállók listája, az oldal legalján pedig el lehet menteni a kiválasztott útvonalat vagy akár ki is lehet törölni, ha már nem használják. Az útvonal-szerkesztő és egy betöltött útvonal a mellékelt 5.9 ábrán látható.

The screenshot displays the 'Bus4U ADMIN' interface for managing bus routes. The left sidebar contains navigation options: Home, Stations, Routes (selected), Schedule, Buses, and Employees. The main area is titled 'Routes' and includes a sub-header 'Select a route from the list below to edit or create a new one'. A dropdown menu shows the selected route 'Gyergyó-Marosvásárhely', with a 'NEW ROUTE' button to the right. Below this is the 'Route editor' section, which features a 'Name' field (Gyergyó-Marosvásárhely), a 'Basic fare' field (0.0), a 'Lei' field, a 'Station' dropdown, and an 'ADD STATION' button. A list of 9 stations is shown, each with a location name, address, coordinates, and a delete icon. At the bottom, there are 'DELETE ROUTE' and 'SAVE ROUTE' buttons.

| Name                                              | Basic fare        | Lei               | Station | ADD STATION |
|---------------------------------------------------|-------------------|-------------------|---------|-------------|
| Gyergyó-Marosvásárhely                            | 0.0               |                   |         |             |
| Stations (9)                                      |                   |                   |         |             |
| Gyergyó Központ<br>Pla. Libertatii nr. 26         | 25.59998843540028 | 46.72080260302078 |         |             |
| Gyergyó Állomás<br>Str. Garii nr. 16              | 25.5740503955354  | 46.71952175752869 |         |             |
| Parajd központ<br>Str. principala nr. 23          | 25.12658622171633 | 46.55101414841867 |         |             |
| Dedeman<br>Bul. 1 Decembrie nr. 189               | 24.59941          | 46.52678          |         |             |
| Plata Trandafirilor<br>Plata Trandafirilor nr. 10 | 24.56029          | 46.54377          |         |             |
| Dedeman<br>Bul. 1 Decembrie nr. 189               | 24.59941          | 46.52678          |         |             |
| Parajd központ<br>Str. principala nr. 23          | 25.12658622171633 | 46.55101414841867 |         |             |
| Gyergyó Állomás<br>Str. Garii nr. 16              | 25.5740503955354  | 46.71952175752869 |         |             |
| Gyergyó Központ<br>Pla. Libertatii nr. 26         | 25.59998843540028 | 46.72080260302078 |         |             |

5.9. ábra. Útvonalak létrehozása, módosítása és törlése

### 5.3.4. Menetrend

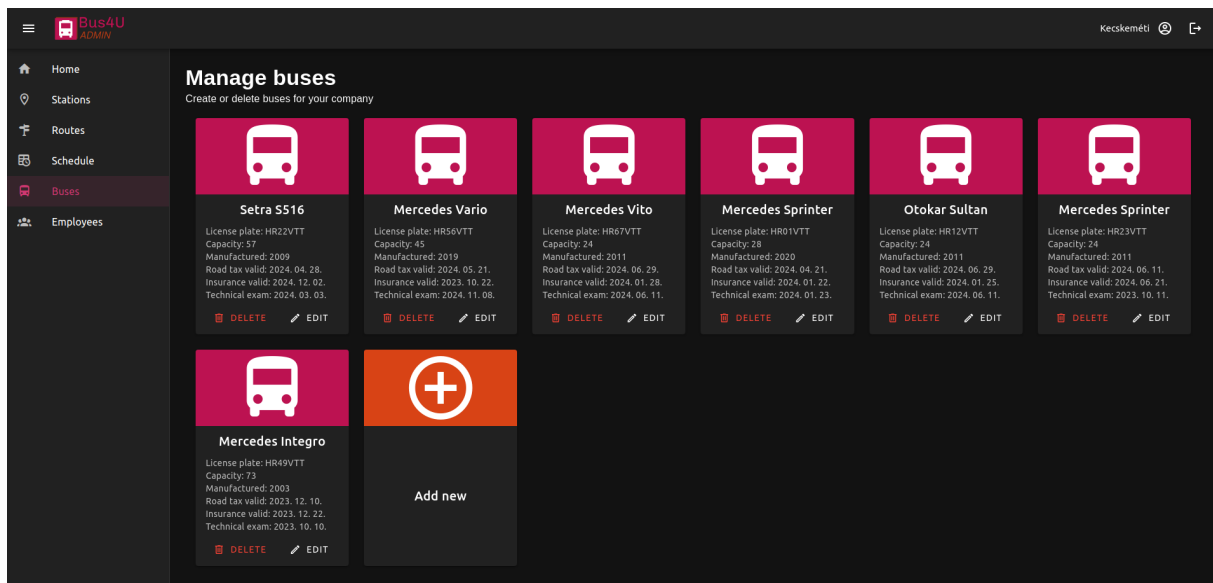
A következő oldal a Menetrend, ahol a létrehozott útvonalak megállóihoz lehet órarendeket hozzárendelni. Itt az útvonal és megálló kiválasztása után meg kell adni az órarendre érvényes napokat, a következő megállóig megszabott jegyárat és az indulási időpontokat. Egy megállóhoz hozzá lehet adni több ilyen órarendet is, így el lehet különíteni a hétvégét a hétköznapoktól, de akár minden napra lehet más-más órarendet is megadni, ha különböznek az indulási időpontok vagy akár a jegyárak. Az órarendeket egyesével lehet szerkeszteni és törölni is, csak az a fontos, hogy a módosításokat a felhasználó az oldal alján található gombbal mentse el, különben nem lesznek érvényesek. A 5.10 ábrán látható egy példa a menetrend szerkesztésére.

The screenshot shows the 'Schedule' page in the Bus4U ADMIN system. The left sidebar contains navigation links: Home, Stations, Routes, Schedule (active), Buses, and Employees. The main content area is titled 'Schedule' with the instruction 'Select a route and a start station to add/edit timetables'. It features three dropdown menus for 'Route' (Városfalva-Udvarhely), 'From station' (Homoródszentpéter), and 'To station' (Homoródszentmárton). Below these, there are two sections for adding or editing timetables. The first section is for 'Monday, Tuesday, Wednesday, Thursday, Friday' with a fare of 3 and times 06:30, 11:45, 14:30, and 17:00. It includes a 'DELETE' button. The second section is for 'Saturday, Sunday' with a fare of 4 and times 10:00 and 14:00. It includes 'CLEAR', 'ADD TIME', and 'SAVE DAY(S)' buttons. At the bottom, there are 'CANCEL EDITING' and 'SAVE SCHEDULE' buttons.

5.10. ábra. Menetrend szerkesztése

### 5.3.5. Buszok és alkalmazottak

Az utolsó két oldalon a buszokat és az alkalmazottakat lehet kezelni. Ez a két oldal hasonlóan néz ki és hasonlóan működik. Mindkét esetben látható egy lista a meglévő buszokkal vagy alkalmazottakkal, ahol minden elemet egy ú.n. kártya komponens reprezentál. A kártyák tartalmazzák az elemek összes tulajdonságát, és két gombot, amelyekkel lehet módosítani vagy törölni őket. Az utolsó kártya segítségével újabb példányt is lehet létrehozni az adott kategóriában. A buszok létrehozásánál meg kell adni a márkát, a rendszámot, a férőhelyek számát, a gyártási évet, valamint az útdó, a biztosítás és a műszaki engedély lejárat dátumait. Az utóbbiakról később emlékeztető fog megjelenni a weboldalon, amikor közeledik azok lejárat ideje. Új alkalmazott hozzáadásánál meg kell adni a nevet, e-mail címet, beosztást (ez adja majd meg a jogait a weboldalon), telefonszámot, lakcímet (ez opcionális), és egy új jelszavat. A buszok listája és a rajta szereplő kártyák a 5.11 ábrán láthatóak, míg a busz szerkesztésére és az alkalmazott létrehozására példa a 5.12 ábrán.



5.11. ábra. Buszok listája

Bus4U ADMIN

Kecskeméti

Edit bus

Update existing bus

Brand

Mercedes Sprinter

License plate

HR23VTT

Capacity

24

Manufacturing date

2011. január

Road tax valid until

2024. 06. 11.

Insurance valid until

2024. 06. 21.

Technical exam valid until

2023. 10. 11.

SAVE

Bus4U ADMIN

Kecskeméti

Add employee

Create new employee account

First Name

Last Name

Email

Role

Phone Number

Address

Password

Password confirmation

CREATE

5.12. ábra. Busz szerkesztése és alkalmazott létrehozása



## 6. fejezet

# Összefoglalás

Befejezésképpen bátran kijelenthetem, hogy a projekt elején leszögezett terveket sikerült implementálni, és fejlesztés közben egyéb hasznos funkcionalitásokat is sikerült hozzáadni a végső eredményhez. Ezáltal a végfelhasználóknak egy-egy funkciódús és jól működő weboldal és applikáció áll a rendelkezésükre, amely nagyban megkönnyíti a tömegközlekedésben való részvételt, bármilyen eszközön is próbálnák elérni ezt. A busztársaságoknak pedig egy egyszerű vezérlőfelület van biztosítva, amely segítségével könnyen és gyorsan tudják kezelni a buszokat, az útvonalakat, a menetrendeket, a jegyárakat és az alkalmazottakat, mindet egy helyen.

Véleményem szerint a Bus4U további idő és munka befektetésével, egy reális piaci rést tud betölteni, mivel a buszokkal való közlekedés elengedhetetlen a forgalom, és a környezet-szennyezés csökkentéséhez, de jelenleg, a rendszer hiányosságai miatt ez egy nem túl népszerű közlekedési eszköz.

### 6.1. További fejlesztési lehetőségek

Felhasználók szempontjából a következő fontos fejlesztés a buszok telítettségének valós idejű követése lenne, mert ezzel elkerülhető lenne a zsúfoltság a buszokon, és így sokat javulna a tömegközlekedés minősége. Be szeretnénk idővel vezetni a bérletek, illetve különböző kedvezmények kezelésének lehetőségét is, mert ez is növelheti a népszerűséget.

A busztársaságok szempontjából még több lehetőség áll rendelkezésre. Mivel a projektet úgy terveztük, hogy több busztársaság is fel tudja használni, így ez jelen pillanatban általános használatra van optimalizálva. Ennek ellenére egy-egy partner cég kérésére személyre is szabhatnánk a rendszert, egyedi funkciókat és kinézetet bevezetve. Az is hasznos lenne például, ha a cég akár a teljes könyvelését le tudná itt bonyolítani: az alkalmazottak munkáóráit automatikusan vezetni, az autóbuszok tankolásait összesíteni, illetve buszokon lévő POS terminál GPS moduljának segítségével akár a Foaie de parcsurs vezetése is automatizálható lenne. A könyveléshez a járművek szervízköltségének számláit is kellene gyűjteni az oldalra, amivel további hasznos kimutatásokat tudnánk generálni.

# Irodalomjegyzék

- [1] Rashi Desai. Google material design—the user interface development notion. In *IJSRD National Conference on Advances in Computer Science Engineering & Technology*, pages 125–7, 2017.
- [2] Pablo David Garaguso. *Vue.js 3 Design Patterns and Best Practices: Develop scalable and robust applications with Vite, Pinia, and Vue Router*. Packt Publishing Ltd, 2023.
- [3] Erik Hanchett and Ben Listwon. *Vue.js in Action*. Simon and Schuster, 2018.
- [4] VIDA ROLLAND and FEHÉR GÁBOR. Infrastrukturális vagy közösségi érzékelés az okos városokban?
- [5] Supercharge. Bkk x supercharge: All-in-one mobility solution. Accessed: 2024-11-09.
- [6] Eric Windmill. *Flutter in action*. Simon and Schuster, 2020.